How should a tool-value cache handle parallel tool calls?

- **Team:** Team 39 — Saifei Liao, Guohao Chen, Duc Anh Than, Troy Wu
- **Advisor:** Bhaskar Kataria — bk478@cornell.edu — GitHub @bhaskar792
- **Compute:** A few A100s (to serve an open-weight coding model)

Background

Reinforcement learning post-training is how LLMs are made better at agentic tasks. GRPO (Group Relative Policy Optimization) is the common recipe: for each prompt the model generates a group of rollouts (say 10), each rollout is scored by a reward function or reward model, and the model's parameters are updated toward the rollouts that scored above the group's average. Agentic tasks involve tools — a calculator, a Python interpreter, a SQL engine, a shell, a web API. During a rollout the model emits a tool call, that rollout pauses while the tool runs, and the result is appended to the context before generation resumes.

Because all rollouts of a prompt start from the same state and the policy is similar across them, the same tool calls recur across rollouts: the same query, the same command, the same expression. How often tools are called, how the calls distribute across tool types, and how long each call takes determine how much of a training step is spent waiting on tools rather than computing on the accelerator — and therefore how much a cache or a smarter scheduler could save.

Modern agents, however, often emit several tool calls in a single turn when the calls are independent. For example, an agent might read three files at once, query two tables, or issue several searches. The harness runs these calls concurrently, waits for all of them, and appends each result matched to its call ID before generation resumes. Agentic benchmarks expose this pattern heavily. A typical trajectory opens with two or three heavy setup calls, such as building a repo, installing dependencies, or dumping a schema, and then fans out into parallel calls. Benchmarks also execute the same task multiple times, across different model checkpoints to evaluate model quality. Since every attempt starts from an identical environment, both the setup calls and many of the parallel calls repeat across attempts.

TVCache as it stands assumes one tool call per turn. It cannot store a turn that contains several calls. It cannot look up a batch where only some calls hit, or return results in the right order with the right call IDs. It also cannot recognize identical calls in flight at the same moment from concurrent attempts. As a result, only the sequential setup phase benefits from caching, and the parallel phase of every attempt runs uncached. No one has measured how common parallel tool calls are across benchmarks, or worked out how a cache should store and retrieve them.

Objective

This project covers inference only (no training).

- Measure how common parallel tool calls are across benchmarks: the fraction of turns with more than one tool call and the CDF of calls per turn, both per benchmark and per tool type. Each benchmark follows the pattern of a few heavy setup calls followed by a parallel fan-out:
  - DeepSWE: Use DeepSWE benchmark and for the environment use the base image without the setup commands in the environment dockerfile. Ask the agent

to setup the codebase by cloning the repo, building the codebase etc and then solve the task. You will have to modify the prompt to include the instruction on environment setup. Run agents on a task multiple trials to determine pass rate.

- Design how to store parallel tool calls in the cache: compare storing each call independently, storing the whole turn as one entry, and a hybrid of the two, covering duplicate calls within a turn, call-ID handling, and calls with side effects.
- Design how to look up parallel tool calls, handling full hits (every call in the turn is cached), partial hits (only the misses are executed), and identical calls in flight at the same time from concurrent attempts.
- Report per-call hit rate, per-turn hit rate, and the tool-wait time actually saved. A partial hit still waits on its slowest miss, so per-call hit rate alone overstates the savings.

Example deliverables

- CDF of calls per turn and the parallel-turn fraction, per benchmark and per tool type.
- A design writeup comparing the storage and lookup options, with the chosen design justified.
- Parallel-call support in the cache server and client, with tests for ordering, call-ID mapping, partial hits, and concurrent duplicates.
- Hit rates and tool-wait time saved under three configurations: no cache, the current single-call cache, and the parallel-aware cache.
- Confirmation that pass@1 and pass@5 are unchanged with the cache enabled.
- Raw logs in a documented format with these fields: tool name, arguments, call ID, turn ID, hit/miss status, timing, attempt, and task.

Code and tools

- tvcache-rust (cache server): https://github.com/abhivijay96/tvcache-rust
- tvclient: https://github.com/abhivijay96/tvclient
- vLLM: https://vllm.ai